

Architecture decision records

For anyone about to change a decision that's expensive or dangerous to reverse. Argue with the "why".

Status

- ADR-001 to ADR-020: accepted, August 2026, Part A. ADR-021 to ADR-028: accepted, August 2026, Part B.
- Retention and RLS access policy are deliberately undecided (ADR-020). Don't invent them.

ADR-001: Attendance comes from a mid-session form, not the Zoom API

- A Google Form released 15 to 25 minutes in asks for a passphrase the teacher says aloud and shows on screen. Verified email supplies identity.
- Since March 2023 Zoom withholds guest `id` and `participant_user_id` and gives email only for signed-in users, so a guest record is a self-typed name plus duration.
- Rejected: a form at lesson start (proves joining only), a timestamp with no passphrase (an idle tab passes).
- Spoken and shown (deaf and hard-of-hearing fellows, dropped audio), so it can leak: one signal among several, never proof alone.

ADR-002: Google-Verified email collection, not a typed address

- Verified only. `template.verify_template` refuses a form whose settings don't read back `VERIFIED`; `TemplateNotVerified` blocks provisioning.
- A typed address looks identical in the database, so self-reported identity would return invisibly.
- Rejected: Responder input, an email question, a roster dropdown.

ADR-003: Template-and-copy, not per-form API settings

- `forms.batchUpdate` → `updateSettings` → `emailCollectionType` has been reported returning `400 INVALID_ARGUMENT` with no working enum value.
- A human sets Verified on one template once; the app refuses until `form.settings` reads `VERIFIED`. Session forms are Drive `files.copy` copies. `batchUpdate` only sets title, description and question text. See `src/cufa/template.py`, `docs/google-api-traps.md` trap 2.
- The app still tries `try_set_verified_email` first and expects the 400, so the manual step vanishes if Google fixes the enum.
- The template is re-verified before every provisioning run, not read from a stored flag, because someone may edit it months later.
- Rejected: catching the 400 and continuing (forms collect nothing), configuring every form by hand (CU has no data manager).

ADR-004: Read `forms.responses.list`, not a linked spreadsheet

- `Form.setDestination()` is Apps Script only, so API-made forms have no linked Sheet.

- Poll with `filter=timestamp > <watermark>`; the per-form watermark advances only after a full successful pass (`src/cufa/ingest/forms_api.py`). Re-reads are free because `source_event_id` makes writes idempotent.
- The API returns `respondentEmail` and RFC3339 UTC timestamps, so the Sheets timezone trap doesn't exist here. CSV stays as a fallback for manually created forms.

ADR-005: User OAuth, not a service account

- A service account can't own a Google Form. One CU staff member grants `forms.body` and `drive.file` once; forms are owned by that CU account.
- `drive.file` is enough because the app creates the template, so it and every copy stay in scope. Full `drive` would reach the whole Drive and trigger Google verification.
- Rejected: domain-wide delegation (needs Workspace super-admin, grants standing impersonation; out of scope for Part A).

ADR-006: Session is derived from the timestamp; the form has no session dropdown

- The form has exactly one question ("Today's passphrase").
- Session comes from `submitted_at_utc` within `[scheduled_at_utc - grace, scheduled_at_utc + duration + grace]` (`ingest/common.py::assign_session`). On the API path the form's own session wins.
- If timestamp and form session disagree, warn (naming the sessions) as a configuration error. Don't reassign the row.

ADR-007: Never drop a submission

- Wrong passphrases, blanks, unknown addresses, and no-window or two-window timestamps all become `checkin` rows, with reasons in `session_match / passphrase_match`. Unknown CSV columns go to `extra_fields`.
- A dropped row can't be recovered, and filters delete exactly the cases worth looking at.
- The `checkin_no_mutation` trigger (`supabase/migrations/..._checkin_immutability.sql`) blocks every DELETE and every UPDATE to an observed column.

ADR-008: The observation is separate from the decision

- `checkin` is immutable (trigger-enforced); `latency_seconds` is the only exempt column (ADR-014).
- `attendance_decision` is append-only and versioned: `decided_by`, `rule_name`, `ai_model`, `ai_prompt_version`, `ai_reasoning`, `human_email`, `note`, `created_at`. Superseding is an UPDATE of `superseded_at` plus an INSERT (`src/cufa/decisions.py::record_decision`).
- A partial unique index on `checkin_id` WHERE `superseded_at IS NULL` allows one current decision, so code that forgets to supersede fails loudly.
- `checkin` stores email, not `fellow_id`. `v_checkin_resolved` resolves identity at read time, so a roster fix re-attributes history with no backfill.

ADR-009: Three tiers: deterministic rules, then a model, then a human

- Tier 1: rules over the recorded comparison (`adjudicate/rules.py`), reproducible from the row alone. Only `mismatch` inside a window goes to tier 2, Gemini (`adjudicate/ai.py`).
- Tier 3: a human (`cufa decide` or the console review screen), `confidence = 1.0`. **Human decisions are never superseded:** `adjudicate` skips them; `--force` overrides only with a warning naming what it overwrites.
- Tier 2 never crashes the run: no key, network or quota writes `status='needs_review'`, `decided_by='rule'`, `rule_name='ai_unavailable'`.
- Rejected: rules only (can't read "justice i think?"), a model on everything (slow, costly, not reproducible), a human on everything.

ADR-010: `needs_review` is never collapsed into `not_attended`

- Undecided cases get `status='needs_review'`, `attended = NULL`; nothing downstream converts that to absent. Constraint `decision_status_matches_attended` enforces the NULL.
- A wrong answer from a verified address in the window usually means mishearing, joining late or a bad session time. The AI tier's "did not hear" also gives `needs_review`.

ADR-011: Fuzzy passphrase matching is on by default

- Normalize first: NFKC, casefold, strip punctuation, collapse whitespace.
- Levenshtein $\leq$ `CUFA_MAX_EDIT_DISTANCE` (default 1) is `fuzzy`: `attended`, confidence 0.9 vs 1.0 for `exact`.
- Rejecting `justise` punishes someone who heard it. Distance 1 can't bridge two words; wider cases go to tier 2. Rejected: `exact` only, a larger default, phonetic tier-1 matching.

ADR-012: Levenshtein is implemented here, not imported

- About 20 lines in `src/cufa/text.py::levenshtein`, with a `max_distance` short-circuit.
- Every dependency is maintenance CU inherits. Rejected: `python-Levenshtein`, `rapidfuzz`, `jellyfish`.

ADR-013: Gemini receives exactly two strings

- Only the expected passphrase and the submitted answer: no names, emails, cohort or history. Extra context could be wrong or count against a fellow.
- `temperature=0`, structured JSON response schema, `PROMPT_VERSION` stored on every decision.
- Cache keyed on (`expected_normalized`, `submitted_normalized`, `prompt_version`, `model`), checked before every call. A prompt change invalidates it.
- Humans sample its judgments with `cufa review --status ai`.

ADR-014: Latency is stored and never interpreted

- Store `latency_seconds`; no thresholds, "suspicious" markers or derived flags. A threshold would be policy CU hasn't written.

- `T0` is `session.announced_at_utc` if the teacher pressed "Announce now", else the earliest matched submission (so the first submitter reads 0); NULL if no session matched.
- Don't clamp negatives: a submission before a late announcement is a real observation.
- `latency.recompute_for_session` recomputes it, which is why it's exempt from the immutability trigger.

ADR-015: The provisioning row is written before publish, so a failure resumes

- The `session_form` row is inserted once the copy exists (including the `batch_update` error path), with `publish_verified_at` NULL. Ready means `publish_verified_at` IS NOT NULL.
- A later `cufa provision` resumes: publishes and verifies the existing form. Ready sessions are skipped. Every attempt writes a `provisioning_log` row.
- `published_at` set with `publish_verified_at` NULL is the half-provisioned state.
- Rejected: writing the row only on success (orphan forms in Drive), deleting partial forms, an in-memory retry loop.

ADR-016: psycpg 3 and plain SQL, not an ORM or `supabase-py`

- `psycpg[binary]`, hand-written SQL in `src/cufa/db.py` and per-module query functions. Schema in `supabase/migrations/*.sql`.
- ORMs hide what the schema relies on: the partial unique index, the trigger exemption, `ON CONFLICT DO NOTHING`, `count(*) FILTER (WHERE ...)`. Doc queries paste straight into Studio.
- RLS is on; the pipeline connects as the service role.

ADR-017: FastAPI + Jinja2 + HTMX, not Next.js on Supabase

- Server-rendered HTML in the same Python codebase, HTMX for interactive bits. One language, one deploy, no build step.
- **Everything the console does is also a CLI command** (`cufa provision`, `cufa pull`, `cufa decide`, `cufa review`, ...), so the system works if the web app breaks.

ADR-018: `--sheet-timezone` is mandatory with no default

- Sheets exports local wall-clock timestamps (`2026-09-15 13:05:00`) with no offset.
- `cufa ingest part-a` requires `--sheet-timezone <IANA>`. Missing it raises `MissingTimezone`, naming the flag, an example command and Sheets → File → Settings → Time zone.
- `zoneinfo` converts at the parser; `submitted_at_raw` and `source_timezone` are stored for re-derivation.
- Every default (UTC, machine zone, inferred) silently shifts rows by hours.

ADR-019: The refresh token is encrypted at rest, in the database

- Fernet, key in env var `CUFA_ENCRYPTION_KEY` only (generate with `python -m cufa.crypto keygen`); ciphertext in `google_credential.refresh_token_enc` (bytea).
- `credential_status()` doesn't decrypt. `disconnect()` clears the ciphertext but keeps the row (who connected, when).

- RLS on `google_credential` with no policy, so only the service role reads it. Logs redact token- and key-shaped strings at every level, DEBUG included.
- `Settings.require_encryption_key()` refuses to store a token without a key.
- Rejected: a token file (gets committed), plaintext, `pgcrypto` (key next to ciphertext), a cloud KMS.

ADR-020: Retention period and RLS access policy are deliberately undecided

- Status: undecided. Part B extends this, doesn't resolve it. Don't invent CU's policy.
- `src/cufa/form_content.py` has a `TODO(retention)` marker, and the form header tells fellows. `supabase/migrations/..._rls.sql` enables RLS on `fellow`, `checkin`, `attendance_decision`, `google_credential` with `TODO(access)` stubs whose predicate is `false`.
- CU wants full-time staff to see data but hasn't set granular permissions; a "marked absent" judgment may deserve tighter access than a raw timestamp.
- Open (listed in the migration): is `fellow` readable by the same people as `checkin`, is a decision more restricted than an observation, do fellows read their own rows and on what claim.
- Part B: `checkin_b` and `peer_shoutout` get the same stub (plus: can the person named read it?). `help_request` sits outside the shared stub (RLS on, no policy, grants revoked from `anon` and `authenticated`) with its own `TODO(retention)`.

ADR-021: Field order comes from completion research, not from visual balance

- Fixed order: easy click, core processing artefact, rotating slot, optional shoutout, help checkbox last. Only the first three are required.
- Multiple-choice openers complete at 89% vs 83%. Early sensitive items raise abandonment of the whole form, and forced answers hurt completion.
- The console shows this rationale where staff add fields.

ADR-022: A 7-point confidence scale, not a 5-point one

- `scaleQuestion`, `low: 1`, `high: 7`, endpoint labels only. Stored as integer 1 to 7, never rescaled or turned into a percentage.
- Views show median and IQR via `percentile_disc`, never a mean, since Likert is ordinal.
- 5-point scales induce interpolation, and this field is graphed (Preston & Colman; Krosnick & Presser).

ADR-023: One rotating question, not every question every week

- Teacher's question on weeks 1, 4, 7, 10; muddiest point on 2, 5, 8; application on 3, 6, 9. Later weeks wrap.
- Schedule in `config/rotation.json`, owned by the Director of Programs.
- Going from three questions to four cuts completion 18%, and fatigue causes straight-lining that corrupts the confidence trend.
- The teacher's question runs most because it's the only unfakeable one.

ADR-024: Answers are resolved through a recorded question-id map

- After provisioning, `forms.get` reads each Part B form and `questionId` → slot goes in `form_question_map`, keyed by form and matched by item index. Question text is snapshotted then.
- A missing or incomplete map **refuses to ingest**: a skipped answer can be re-pulled, a mis-attributed one can't be detected.
- Rejected: matching by title (rotates, teachers retitle), by payload position, reusing template ids, skipping unknown ids.
- Measured August 2026: `files.copy` preserves ids, so the rotating slot keeps one `questionId` with different weekly text. The per-form map and `question_text` snapshot are necessary.
- Tests still run the fake client's `question_id_scheme` under both `preserve` and `regenerate`.

ADR-025: The help checkbox requires a named recipient, or it is not on the form

- `config/help_routing.json` names the recipient; it ships naming the Director of Programs. CU has no fellow-support role (confirmed 2026-08-10).
- Nobody named: provisioning omits the field and logs it, shows it on the session screen and records it in provisioning history.
- Requests are emailed immediately on ingest. The row is written first, so a mail failure loses nothing; the console is the durable channel.

ADR-026: The help checkbox is excluded from every signal, permanently

- Separate `help_request` table, strictest RLS, its own console access list, no granting policy.
- Not in any report, export, aggregate or participation computation; never sent to AI; never logged at any level.
- Two tests: every export and report path runs on data with a request; the SQL each participation, report and export path executes is inspected and fails if it mentions the table.
- A column rides along with every `SELECT *`. If fellows suspect ticking the box costs them, they stop.

ADR-027: The AI clusters content, never people

- Gemini's only Part B job: cluster muddiest-point answers into 2 to 5 themes for the teacher, from anonymous strings only.
- Free text is counted, never graded (grading penalizes ESL and neurodivergent fellows). Straight-lining is a data-quality flag, not a participation signal.
- No API key gives no themes and a clear message, never a failed run. Regenerating supersedes.
- The teacher view is built; a fellow-facing view isn't, pending the data owner's decision.

ADR-028: Shoutouts are collected and resolved, never ranked

- Names are split, normalized and matched within the cohort roster. One unambiguous match links; **ambiguity always goes to a review queue**, recording the resolver.

- A name matching nobody is legal (guests, teachers). A wrong link is invisible, an unlinked one sits in the queue.
- Shoutouts are third-party data: same protection as the submitter's, never shown to the person named without a data-owner decision.
- No leaderboard, ranking, points, streaks or public display. If ever built, rank by recognition given, not received.

ADR-029: Simulated state is detected before Google is asked, and the demo will not overwrite a real install

- Form ids carry provenance (`fake-form-` isn't a real Google id). A stored id from the other client kind is caught before any API call, with a message naming the account and both fixes, not Google's bare `404 Requested entity was not found`.
- `cufa template replace --part a|b` retires a template and creates a new one, deactivating rather than deleting. It's explicit because a template needs the manual Verified step.
- Stale fake session forms heal automatically. A real form that 404s is never discarded; it may be in Drive's bin with responses.
- `make demo` refuses when the database has a connected account, a real template or check-ins, unless `CUFA_DEMO_FORCE=1`.
- Fix: callers' rollbacks were deleting the ADR-015 provisioning row. It's now written on a separate autocommit connection.